

Programming Assignment 3

Due at the beginning of your discussion session on
September 15-17, 2026

Reading

In addition to the following topics, the quiz syllabus includes any material covered in the lectures:

- Section 16.1 (“Normal Loop-With-Exit Loops” only) in Code Complete
- Section 16.2 (Except “Enter the loop from one location only”, “Checking Endpoints”, “Using Loop Variables”, and “How Long ...”) in Code Complete.
- Section 19.6 in Code Complete
- Item 58, 59, and 67 in Effective Java.

Programming

First, make all the changes discussed in your discussion section. Additionally, you should refactor your code to make sure that it adopts the principles covered in the reading assignments. You should modify your code to avoid methods with high McCabe’s complexity.

In this programming assignment, you will implement arithmetic operations on matrixes and error handling.

Error Handling

As a reminder, the following rules apply to error handling.

Exceptions and Assertions

Error checking will be a topic of a future lecture. For the time being, the following guidelines should be implemented:

- If a method is *visible* outside the package (e.g., public methods in public classes), failed pre-conditions should cause the code to throw an appropriate exception. An exception can carry supplemental information that is passed as the constructor's argument(s).
- If a method is *invisible* from outside the package (e.g., private, or package-private methods), failed pre-conditions should be verified by an `assert`. An `assert` can specify supplemental information as the expression following the colon.

Supplemental Information

In both cases, the error handling should be supplemented by additional information. In Java, the supplemental information can take the form of one or more of the following:

- *Nothing*, if the source of the error is clear from other information already available (such as a stack trace).
- A `String` that outlines the reasons for the error. Strings should only be used if their use conforms to Item 62 in the reading assignment.
- A `Throwable` that describes the cause of the error.

Null Arguments

A common occurrence of failed pre-conditions are arguments that are *null* even though null parameters are not expected or meaningful. Unless stated otherwise, you should assume through this series of assignments that null parameters constitute a failed pre-condition that necessitates error handling.

The Java Language Features module on canvas contains pointers on assertions, exceptions, and null parameter checking.

Entry Map

Although the `get` and `getOrDefault` methods allow us to access individual entries, it is often helpful to access, iterate over, or modify the whole matrix. Define the public method

```
Stream<Entry<T>> stream()
```

that returns a stream of the matrix entries. Define the method

```
EntryMap<T> remap(  
    Function<Coordinates, Coordinates> coordinatesMapper,  
    Function<T, T> valueMapper)
```

that returns a new `EntryMap` in which each coordinate is replaced by the application of the `coordinatesMapper` and each value by the application of the `valueMapper`.

In Strassen's algorithm, matrixes must always have size that is a power of 2. The size requirement can be achieved by padding a matrix with additional zero entries. For example, the matrix

$$\begin{pmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 6 & 7 & 8 \end{pmatrix}$$

is equivalent to the matrix

$$\begin{pmatrix} 1 & 2 & 3 & 0 \\ 5 & 6 & 7 & 0 \\ 6 & 7 & 8 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

which has size 4. Padding with zeroes makes no changes to the private `NavigableMap` because the `entryMap` does not maintain zero entries. Padding only requires a different computation of the matrix size. Define a public `sizeRounded()` method that returns the matrix size rounded to the next power of 2.

Matrix

A numerical matrix is represented by an `EntryMap` of floating-point numbers that implements additional operations for adding, subtracting, and multiplying matrixes. Define a public final class `Matrix` that contains a private final `EntryMap<Float>` representation whose value is set by a *private* constructor. Matrixes are instantiated by the methods:

```
public static Matrix from(EntryMap<Float> entryMap)
```

```
public static Matrix from(  
    Map<Coordinates, Float> entryMap)
```

The size of a matrix is the size of its representation rounded to the next power of 2. Individual values can be accessed with the public method `get(Coordinates coordinates)`. The public method `get()` is provided for convenience and returns the value at the

ORIGIN. The public method `Stream<Entry<Float>> stream()` is delegated to the representation.

The basic arithmetic operations are accomplished by the following public methods:

- `negated()` returns a new `Matrix` in which all values have been reversed in sign
- `plus(Matrix other)` and `minus(Matrix other)` returns a new matrix containing the result of the given operation.

The `plus` and `minus` method throw an exception if the other's size differs from the matrix size, as described next.

Exceptions

The `InconsistentSizeException` is thrown whenever an operation is attempted on matrixes of differing sizes. Define a public final `InconsistentSizeException` class that extends the standard Java `Exception`. Its `serialVersionUID` can be set to any arbitrary value. It contains two private final integers: the `referenceSize` and the `otherSize`, with public getters, and whose value is set by the public constructor. The `InconsistentSizeException` defines a

```
public static method void validate(  
    int referenceSize, Matrix otherMatrix)
```

that throws an `IllegalArgumentException` whose cause is an appropriate `InconsistentSizeException` if the `otherMatrix` size is different from the reference. The `validate` method should be used in `Matrix::plus` and `Matrix::minus` to validate that the other matrix is not null and of the right size.

The `Matrix` class uses a representation based on floating-point numbers. It is also helpful to create alternative representations in blocks. To this end, first implement the following public methods in the `Coordinates` and `Entry` records:

- `boolean isInRows(int lower, int upper)`
- `boolean isInColumns(int lower, int upper)`

that return whether the coordinate is within the lower (inclusive) and upper (exclusive) limit. These methods can be useful both in general and to check whether an entry belongs to a certain block. Then, implement the `Matrix` method

```
public Matrix subMatrix(
    Coordinates origin,
    Coordinates bound)
```

that returns a new matrix equal to the block from the origin (inclusive) to the bound (exclusive coordinates). For example, the submatrix of

$$\begin{pmatrix} 1 & 2 & 3 & 0 \\ 5 & 6 & 7 & 0 \\ 6 & 7 & 8 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

from (2, 2) to (4, 4) is

$$\begin{pmatrix} 8 & 0 \\ 0 & 0 \end{pmatrix}$$

General Considerations

These classes may contain as many auxiliary private and package-private methods as you see fit, and additional package-private helper classes may be defined. However, any modification or addition to public classes and methods must be approved by the instructors at the discussion board.

You should write JUnit tests to make sure that your primary methods work as intended. However, we will revisit testing later in the course, so extensive testing is not yet recommended. If you have never used JUnit, the JUnit module on canvas lists resources to get you started. Similarly, your code should have a reasonable number of comments, but documentation is going to be the topic of a future assignment. As a general guideline at this stage of the course, comments and tests should be like those accepted in EECS 132. Additionally, comments should only be applied to the code sections that you feel are not self-documenting.

Discussion Guidelines

Although the discussion can range through the whole reading assignment, the emphasis will be on:

- Functions that exceed McCabe's complexity of 4 (if any)
- Non-structured programming constructs and break statements (if any)

Submission

Submission instructions will be posted shortly.

Grading Guidelines

Advance Warning: starting with Programming Assignment 4, an automatic C (or less) is triggered by any routine with complexity greater than 4 or by any substantially repeated piece of code.

